Module: COMP3222 Machine Learning
Assignment: Ensemble Selection

Set by: Tony Bagnall (A.J.Bagnall@soton.ac.uk)
Checked by: Kate Farrahi (K.Farrahi@soton.ac.uk)
Date set: Thursday 15th October 2026
Value: 20%

Date due: Wednesday 11th November 2026 5pm
Returned by: 11th December 2026
Submission: Moodle

Learning outcomes

Students will learn how heterogeneous classifier ensembles can be constructed from a pool of base classifiers, how classifier performance can be estimated using out-of-fold predictions, and how a greedy ensemble selection algorithm can be used to choose a subset of classifiers. They will also gain experience comparing alternative ensemble strategies and implementing a scikit-learn compatible classifier.

Specification

Overview

Ensemble methods combine the outputs of several classifiers to make a final prediction. One approach is simply to include all available classifiers and combine their predictions using voting or probability averaging. However, adding another classifier does not necessarily improve an ensemble. A classifier may make very similar errors to classifiers already in the ensemble, or its predictions may reduce the quality of the combined prediction.

An alternative is an *overproduce-and-select* strategy. We first construct a pool of candidate classifiers and estimate their predictions on the training data. We then select a subset of classifiers to form the final ensemble. The aim is not simply to find the individually most accurate classifiers: a classifier that is weaker on its own may still improve an ensemble if it makes different errors.

In this coursework you will implement a simple greedy forward ensemble selection algorithm. Starting with the best individual classifier, your algorithm will repeatedly consider adding each remaining classifier to the current ensemble and select the classifier that gives the greatest improvement. Selection will be based only on predictions generated from the training data using cross-validation.

Finally, you will compare your selected ensemble with simpler and more general alternatives, including combining the complete classifier pool and the scikit-learn stacking classifier. The purpose of this final comparison is to introduce some of the issues involved in classifier evaluation. A more complete experimental evaluation will form the final part of the module coursework.

You may use standard scikit-learn estimators and utilities throughout this coursework. You must not use an existing implementation of greedy ensemble selection or another library that performs the selection stage for you.

The datasets, the classifier pool used in Part 3 and the required Python and package versions are provided in the `coursework` folder of the course GitHub repository, whose `README.md` explains how to set up your project. Please use exactly the filenames, function names, class

names, parameters and fitted attributes specified below: your submission will be run against our own tests as well as yours.

General requirements

- `X` is a two-dimensional NumPy array of numeric features and `y` is a one-dimensional NumPy array of class labels. Class labels may be numeric or strings.
- `estimators` is a list of `(name, estimator)` pairs, where each name is a unique string and each estimator is an unfitted scikit-learn compatible classifier that implements `predict_proba`. The pool may contain any such classifier, so rely only on the standard scikit-learn estimator interface.
- The estimator objects in `estimators` are templates. They must never be fitted or modified, and you must not change their random states. Fit fresh clones instead.
- All probability arrays returned by your code use the column order `np.unique(y)`. Wherever a class is chosen from a probability vector, including when calculating accuracy, a tie between classes goes to the class that comes first in this order.
- Invalid arguments must raise a `ValueError` (or a `TypeError` for an argument of the wrong type), never produce output silently. This includes an empty pool, duplicate names, an estimator without `predict_proba`, `X` and `y` of different lengths, a `cv` that is not an integer of at least 2, and a `max_ensemble_size` that is neither `None` nor a positive integer.
- Given the same data, the same arguments and an integer `random_state`, your code must produce identical results on every run, provided the supplied estimators are themselves deterministic.
- All tests will be run with `python -m pytest` from the directory that contains `solution/` and `tests/`.

Generating Out-of-Fold Predictions (5 marks)

Create the file

`solution/pool.py`

and implement

```
def generate_oof_probas(
    estimators,
    X,
    y,
    cv=5,
    random_state=None,
):
    ...
    return oof_probas, scores
```

The function uses k-fold cross-validation, with k given by the integer `cv`, to obtain an out-of-fold class-probability vector for every training case from every classifier. Every stored probability vector must come from a classifier that was not trained on that case, and every case must receive exactly one probability vector from each classifier. Choosing an appropriate cross-validation strategy for classification is part of the task. `random_state` controls how cases are assigned to folds; `random_state=None` leaves the fold assignment unseeded.
The function returns a tuple `(oof_probas, scores)`, where

- `oof_probas` is a NumPy array of shape `(n_estimators, n_cases, n_classes)` containing the out-of-fold class probabilities, with estimators in the order of `estimators` and columns in the order `np.unique(y)`;
- `scores` is a NumPy array of shape `(n_estimators,)` containing the out-of-fold accuracy of each classifier, in the same order as `estimators`.

Your implementation should handle edge cases in the cross-validation procedure appropriately. Create

`tests/test_pool.py`

and write tests that demonstrate that `generate_oof_probas` behaves correctly. Your tests must include at least one test showing that the returned predictions are genuinely out-of-fold. Deciding what else needs testing is part of the task.

Greedy Forward Ensemble Selection (10 marks)

Create the file

`solution/ensemble_selection.py`

and implement

```
class GreedyEnsembleSelector(ClassifierMixin, BaseEstimator):
```

with constructor

```
def __init__(
    self,
    estimators,
    cv=5,
    max_ensemble_size=None,
    metric="accuracy",
    random_state=None,
):
```

The ensemble combines classifiers by averaging their predicted class probabilities with equal weight. The class must follow standard scikit-learn conventions: the constructor only stores its arguments, `fit(X, y)` returns `self`, and calling `predict` or `predict_proba` before `fit` raises `sklearn.exceptions.NotFittedError`.

Selection Algorithm

`fit(X, y)` first obtains the out-of-fold probabilities of every classifier in the pool by calling `generate_oof_probas` with the selector's `cv` and `random_state`. Selection uses only these out-of-fold probabilities.
The *score* of a set of classifiers is the value of `metric` for the average of their out-of-fold probabilities. Greedy forward selection then works as follows. The ensemble starts as the single classifier with the best score. At each subsequent step, every classifier not yet in the ensemble is considered as an addition, and the one giving the best score for the enlarged ensemble is added, provided that score is *strictly* better than the score of the current ensemble. Selection stops when no remaining classifier gives a strict improvement, when every classifier has been selected, or when the ensemble contains `max_ensemble_size` classifiers. If `max_ensemble_size` is `None` there is no limit. A classifier may be selected at most once.
Whenever two classifiers give the same score, including when choosing the first member, select the one that occurs first in `estimators`.
For example, suppose classifier 4 is the best individual classifier. The first stage of selection is

$$\{C_4\}.$$

The next stage considers

$$\{C_4, C_1\}, \quad \{C_4, C_2\}, \quad \{C_4, C_3\}, \quad \ldots$$

and adds whichever candidate gives the greatest improvement in out-of-fold ensemble score.

The `metric` parameter

`metric` must support the values `"accuracy"`, the default, and `"log_loss"`. For `metric="log_loss"`, the score is the multiclass log loss calculated from the averaged out-of-fold probabilities, with columns in the order of `classes_`. Accuracy is maximised and log loss is minimised, so for log loss a strict improvement means a strict decrease. You may support further metrics of your own choice; if you do, list them in the class docstring and explain why you chose them. An unsupported value must raise a `ValueError`.

Fitted Attributes

After fitting, your classifier must store:

- `classes_`: a NumPy array of the class labels, equal to `np.unique(y)`;
- `selected_indices_`: the positions in `estimators` of the selected classifiers, in the order in which they were selected;
- `selected_names_`: the corresponding names;
- `selection_scores_`: the out-of-fold score of the ensemble, in the units of `metric`, after each classifier was added;
- `estimators_`: fresh clones of the selected classifiers, in selection order, each fitted on all of the training data.

The last four attributes must be Python lists of equal length. You may store additional attributes as needed for your implementation.

`predict_proba(X)` and `predict(X)`

`predict_proba(X)` returns the mean of the probabilities predicted by the classifiers in `estimators_`, as an array of shape `(n_cases, n_classes)` with columns corresponding to `classes_`.
`predict(X)` returns, for each case, the class with the highest mean probability, using the tie rule given in the general requirements.
Create

`tests/test_ensemble_selection.py`

and write tests that demonstrate that your selector behaves correctly. Deciding what needs testing is part of the task.

Comparing Ensemble Strategies (5 marks)

You will now perform a small controlled comparison of alternative ways of using the same classifier pool.
This is not intended to be a complete experimental study. Its purpose is to demonstrate that your implementation works on real classification problems and to consider some of the choices involved when constructing an ensemble. More extensive classifier evaluation will be covered later in the module and assessed in the final coursework.
The datasets are eight univariate time series classification problems from the UCR archive, each with a fixed train/test split. The course GitHub repository provides a module `part3_setup.py` that defines:

- `DATASETS`, the list of dataset names to use, in the order they should appear in your results;
- `load_dataset(name)`, which returns `X_train, y_train, X_test, y_test` as NumPy arrays, with one row per time series;
- `make_pool()`, which returns a new list of `(name, estimator)` pairs: the classifier pool;
- `make_stacking()`, which returns an unfitted `StackingClassifier` built from the same pool;
- `CV` and `RANDOM_STATE`, the values to use wherever your code takes `cv` or `random_state`.

The datasets are included in the `coursework/data` folder of the course GitHub repository, and the documentation in `part3_setup.py` explains where it looks for them. Import these definitions rather than copying or changing them, so that every student uses the same data, pool and settings. Place `part3_setup.py` and the `data` folder in the directory that contains `solution/`, as described in `coursework/README.md`, but do not submit them: we will use our own copies.
Create

`solution/compare.py`

which, when run as `python -m solution.compare` from the directory that contains `solution/` and `part3_setup.py`, compares the following approaches on every dataset. All selection and model fitting must use the training data only, and accuracy is measured on the test data.

1. **Best single classifier**: the pool member with the highest out-of-fold accuracy, as given by `generate_oof_probas` on the training data, refitted on the complete training data. A tie goes to the member that comes first in the pool.
2. **Full-pool ensemble**: every pool member fitted on the complete training data, with predicted probabilities averaged.
3. **Stacking**: the classifier returned by `make_stacking()`.
4. **Greedy selection by accuracy**: your `GreedyEnsembleSelector` with `metric="accuracy"`.
5. **Greedy selection by log loss**: your `GreedyEnsembleSelector` with `metric="log_loss"`.

The script must write `results.csv` to the directory it is run from, with one row per dataset, in the order of `DATASETS`, and exactly these columns:

```
dataset,best_single,full_pool,stacking,greedy_accuracy,greedy_log_loss,
n_selected_accuracy,n_selected_log_loss,oof_accuracy
```

(as a single line). The five columns after `dataset` are test accuracies, given to four decimal places. `n_selected_accuracy` and `n_selected_log_loss` are the numbers of classifiers selected by each greedy selector. `oof_accuracy` is the final out-of-fold accuracy of the greedy selector that uses accuracy, that is, the last entry of its `selection_scores_`, to four decimal places.
With the supplied pool and datasets, the script should complete in under about one hour on a typical laptop. We will run your `compare.py` and check that it reproduces your submitted `results.csv`, allowing for small numerical differences between machines.
Also submit

```
answers.md
```

containing concise answers to the following questions, based on your own `results.csv`. Each answer should be no more than approximately 150 words, and should refer to specific datasets in your results.

1. Choose a dataset where greedy selection by accuracy and the full-pool ensemble give noticeably different test accuracy. Using the classifiers your selector chose, explain why the difference might have arisen.
2. Compare selection by accuracy with selection by log loss across your results. How does the choice of metric affect which classifiers are selected, and how many?
3. Compare `oof_accuracy` with `greedy_accuracy`, the test accuracy of the same selector. Is the out-of-fold accuracy a reliable estimate of how the selected ensemble will perform on new data? Explain why or why not.

You are not expected to perform statistical significance testing, extensive hyper-parameter optimisation or additional dataset analysis for this part of the coursework.

Submission Requirements

Submit a single zip file containing:

```
solution/
    pool.py
    ensemble_selection.py
    compare.py

tests/
    test_pool.py
    test_ensemble_selection.py

results.csv
answers.md
```

Your code should include clear docstrings for all public functions, classes and methods, describing their purpose, parameters and return values. Where you made a design choice that this specification leaves open, such as your cross-validation strategy, your handling of edge cases or your additional metric, state the choice and your reason for it in the relevant docstring or comment. You are not expected to produce separate documentation.
Do not submit `part3_setup.py`, the datasets, virtual environments, IDE configuration directories, downloaded packages or generated files other than `results.csv`.
Your code must run using the Python and package versions specified in the course GitHub repository.

Demo and Viva

You will demonstrate your submission after the deadline during scheduled lab classes. You will be asked to run your code and answer questions about your implementation, design choices and results.
The questions may differ between students and may relate specifically to the code you submitted.

Marking

The work contributes 20% of the overall module mark and will be marked out of 20:

1. Generating out-of-fold predictions: 5 marks.
2. Greedy forward ensemble selection: 10 marks.
3. Comparing ensemble strategies: 5 marks.

Marks within each section will be awarded for correctness, completeness, style, appropriate testing and understanding demonstrated in the submitted work and viva. Correctness will be assessed partly by running our own tests against your code. The quality of your tests will be assessed by how effectively they detect incorrect implementations of the specified behaviour. For this, your tests are run against our own implementations, so only tests that use the public interface and behaviour defined in this specification count. Tests of behaviour the specification leaves open, such as your own fold assignment, edge-case choices, additional metrics or private helper functions, are still welcome, but any test that fails on a correct implementation is excluded from this assessment.

Use of Generative AI

You may use generative AI tools to support your work, for example to explain concepts, help debug code, or suggest tests. You may include AI-assisted code in your submission, provided you understand every part of it. You may be asked in the demo/viva to explain, modify or justify any part of your submission.
The answers in `answers.md` must be written in your own words. You may use AI tools to help you understand the concepts, but not to draft the answers.
You must not:

- submit code or text that you do not understand;
- report results that were not produced by running your submitted code;
- include tests that do not genuinely exercise your implementation.

At the end of `answers.md`, include a short statement listing any generative AI tools you used and what you used them for. If you did not use any, say so.
Responsibility for the correctness of your submission remains with you. If the demo/viva shows that you do not understand part of your submission, the marks awarded for that part may be reduced.